Universal Asynchronous RX/TX

Department of Electrical and Electronics Engineering,
UTP

3/17/2014

Adapted from

Richard Kuo

Assistant Professor

National Taiwan Ocean University

OutLine

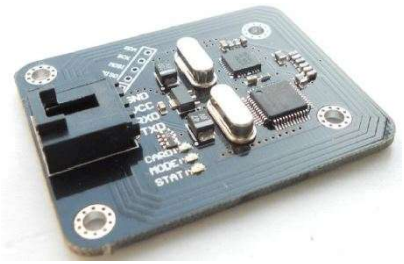
Department of Electrical Electronic Engineering, UTP

9.NuMicro_UART.ppt

- **UART Interface Introduction**
- **Exercise：UART Basic Transmission (Smpl_DrvUART)**
- **Exercise : RS485 Basic Transmission (Smpl_RS485)**
- **Exercise : UART connecting to Ultrasound Sensor (Smpl_UART0_SRF04)**
- **Exercise：UART connecting to Bluetooth module (Smpl_UART0_HC05)**
- **Exercise : UART connecting to Bluetooth module & I2C to read MPU6050 (Smpl_UART0_HC05_MPU6050)**
- **Exercise：UART0 connecting to Bluetooth module, UART1 connecting to US100/SRF04 (Smpl_UART_HC05_US100)**
- **Exercise : UART0 connecting to RDID reader**







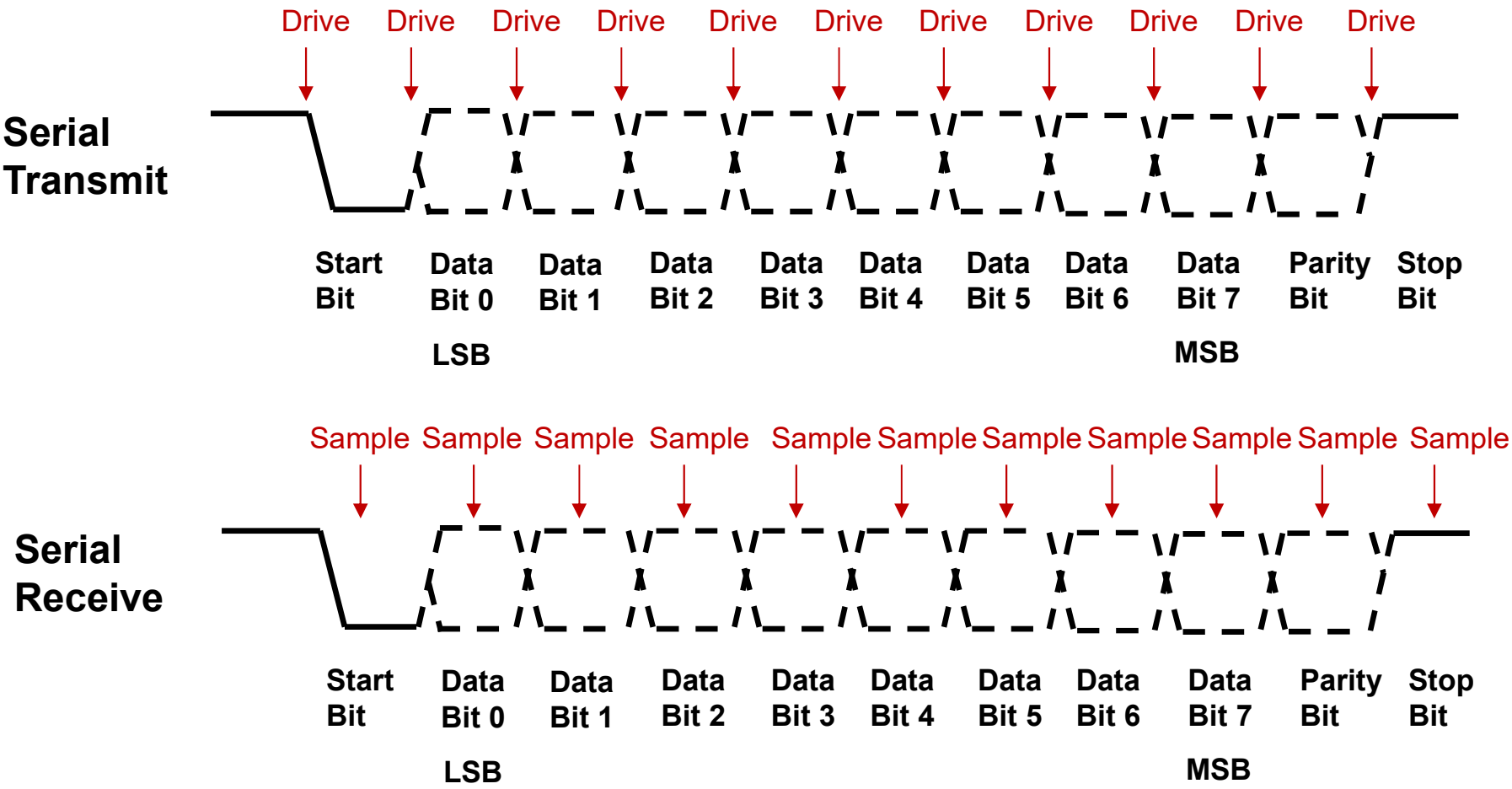
UART pins

Based on NUC140 LQFP 100pin

- UART0
  - RX0/GPB0 : pin32
  - TX0/GPB1 : pin33
- UART1
  - RX1/GPB4 : pin19
  - TX1/GPB5 : pin20
- UART2
  - RX2/GPD14 : pin38
  - TX2/GPD15 : pin39

UART (Universal Asynchronous Receiver Transimitter)

Department of Electrical Electronic Engineering, UTP



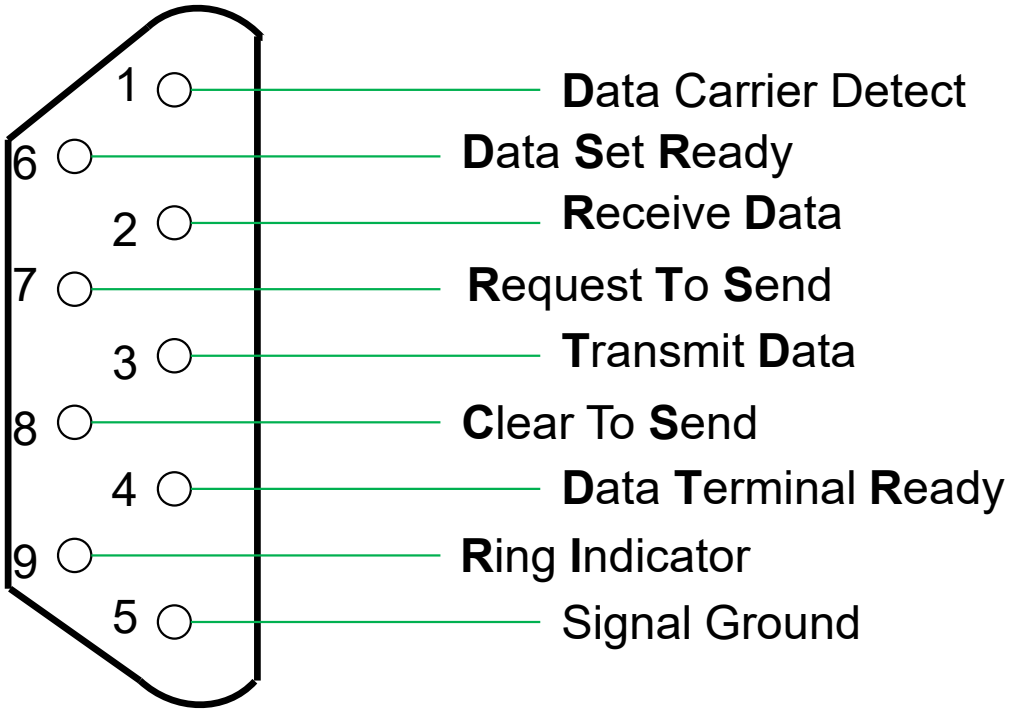
UART Baud Rate : Cycle Time & Interrupt

Department of Electrical Electronic Engineering, UTP

Baud Rate	Cycles per Timer Tick (at 3.57MHz)	Worst-Case Interrupt Cycle Count	Bandwidth Used by UART (%)
9600	372	19	5.1%
19200	186	19	10.2%
28800	124	19	15.3%
57600	62	19	31.6%
115200	31	19	61.3%

RS-232 DB9 Connector

Department of Electrical Electronic Engineering, UTP

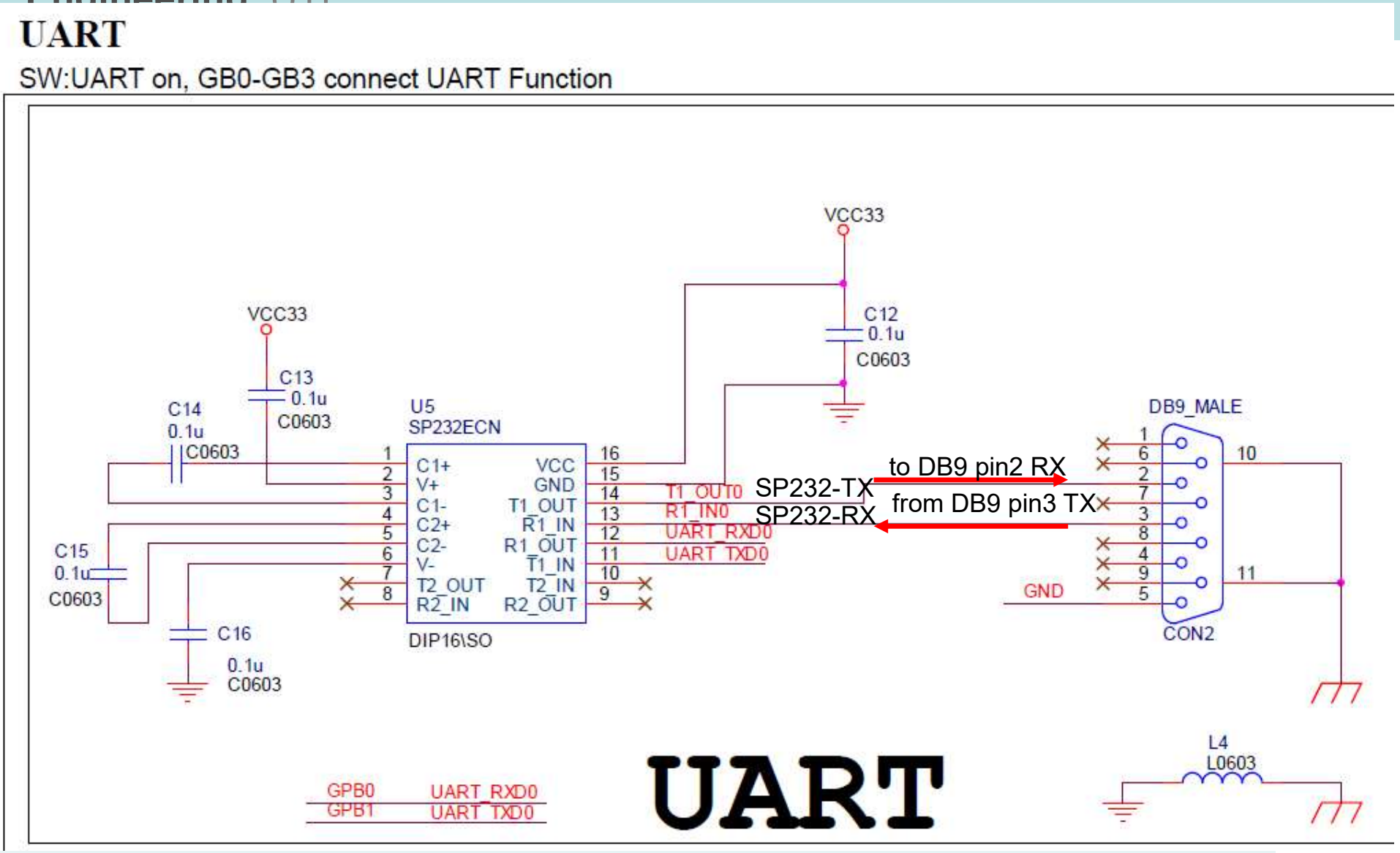


RS-232/422/485 Comparison

	RS-232	RS-422	RS-485
No. of Driver	1	1	32
No. of Receiver	1	10	32
Mode of Operation	Half duplex Full duplex	Half duplex	Half duplex
Max. Distance	15m	1200m	1200m
Max. Speed at 12m	20Kbps	10Mbps	35Mbps
Max. Speed at 1200m	1Kbps	100Kbps	100Kbps
Min. Output Voltage	+-5V	+-2.0V	+-1.5V
Max. Output Voltage	+-25V	+-6V	-7~12V
Receiver Input Range	+-15V	+-10V	-7~12V

UART schematic

Department of Electrical Electronic Engineering UTP

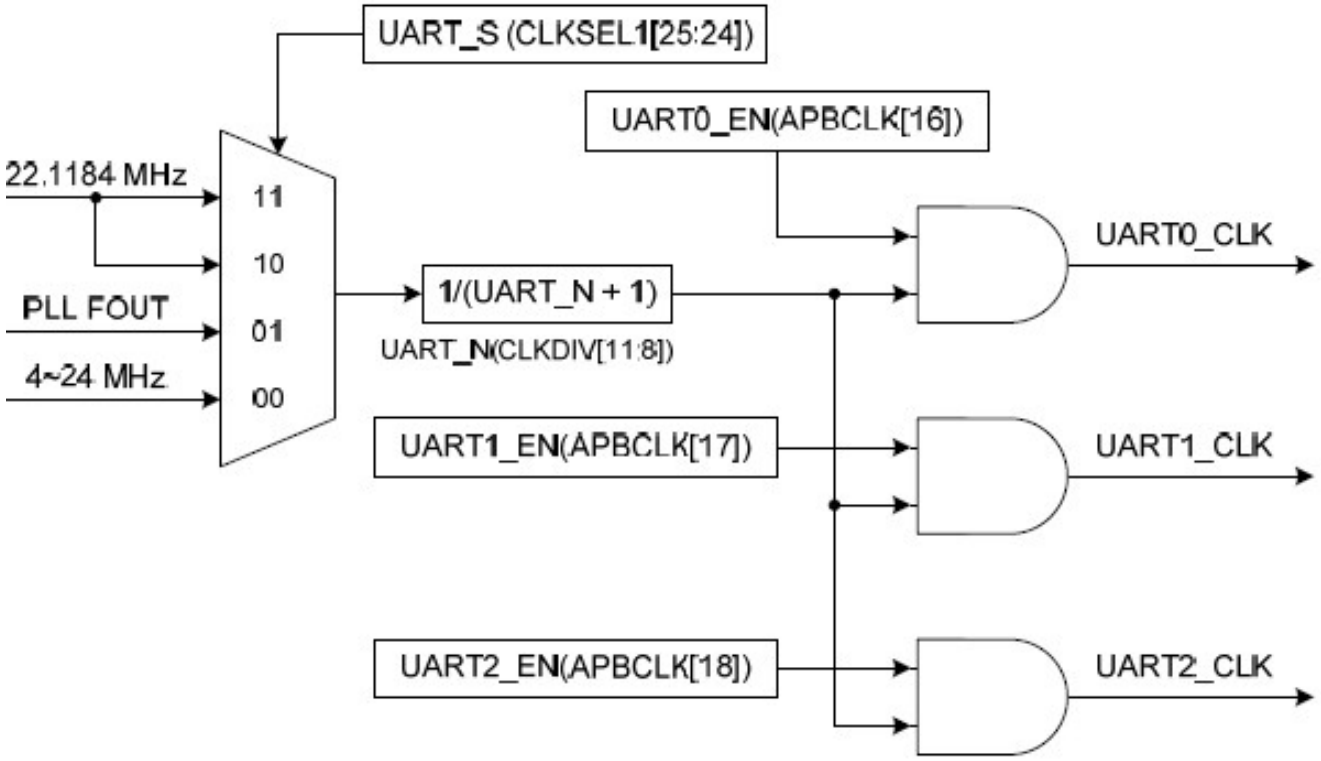


NuMicro MCU UART function

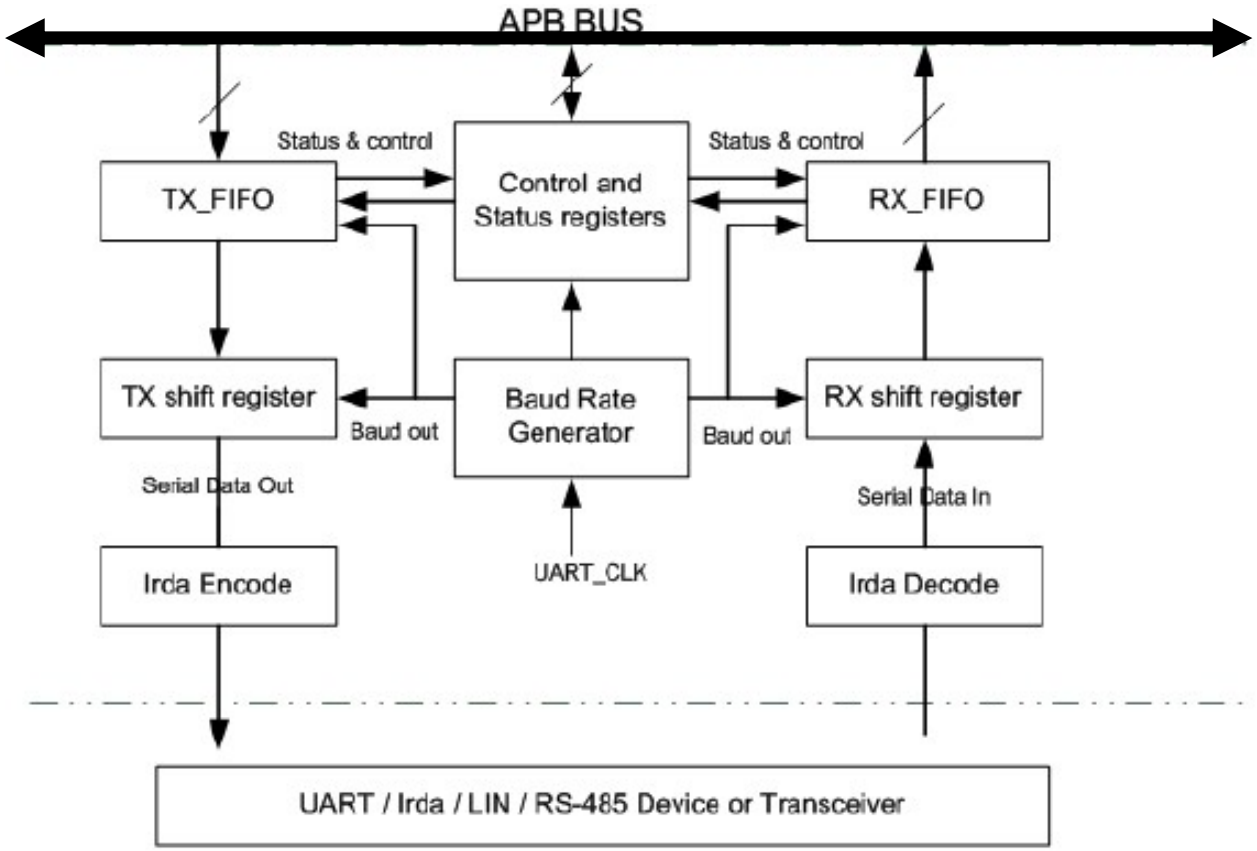
- Provide up to 3 UART ports
  - programmable baud-rate
  - programmable serial-interface
  - Built-in TX/RX buffers (FIFO)
    - UART0 FIFO = 64 bytes
    - UART1 FIFO = 16 bytes
    - UART2 FIFO = 16 bytes
- Support hardware flow-control (/CTS及/RTS) on UART0 & UART1
- Support IrDA SIR function (Infrared transmission use)

UART Clock Selection

Department of Electrical Electronic Engineering, UTP



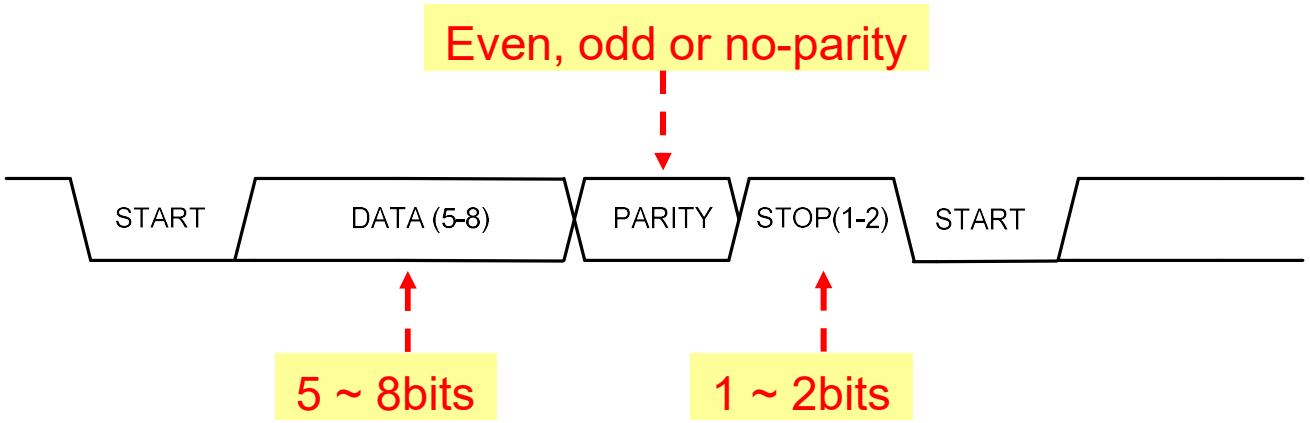
UART Block Diagram

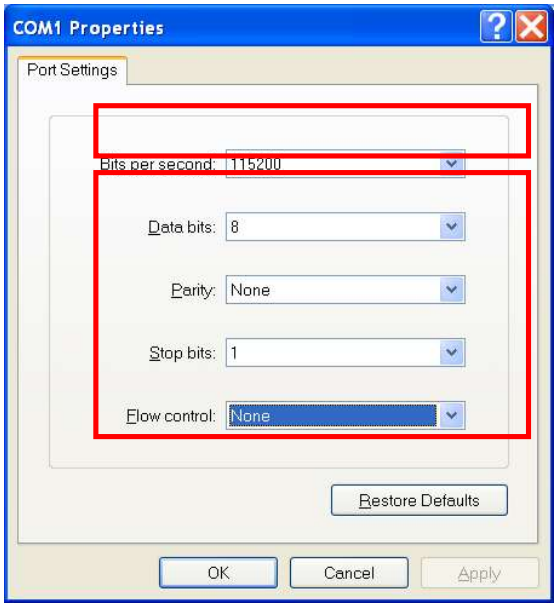


Note : UART0 equips 64 bytes FIFO, UART1/UART2 equip 16 bytes FIFO

UART Setting

Department of Electrical Electronic Engineering, UTP





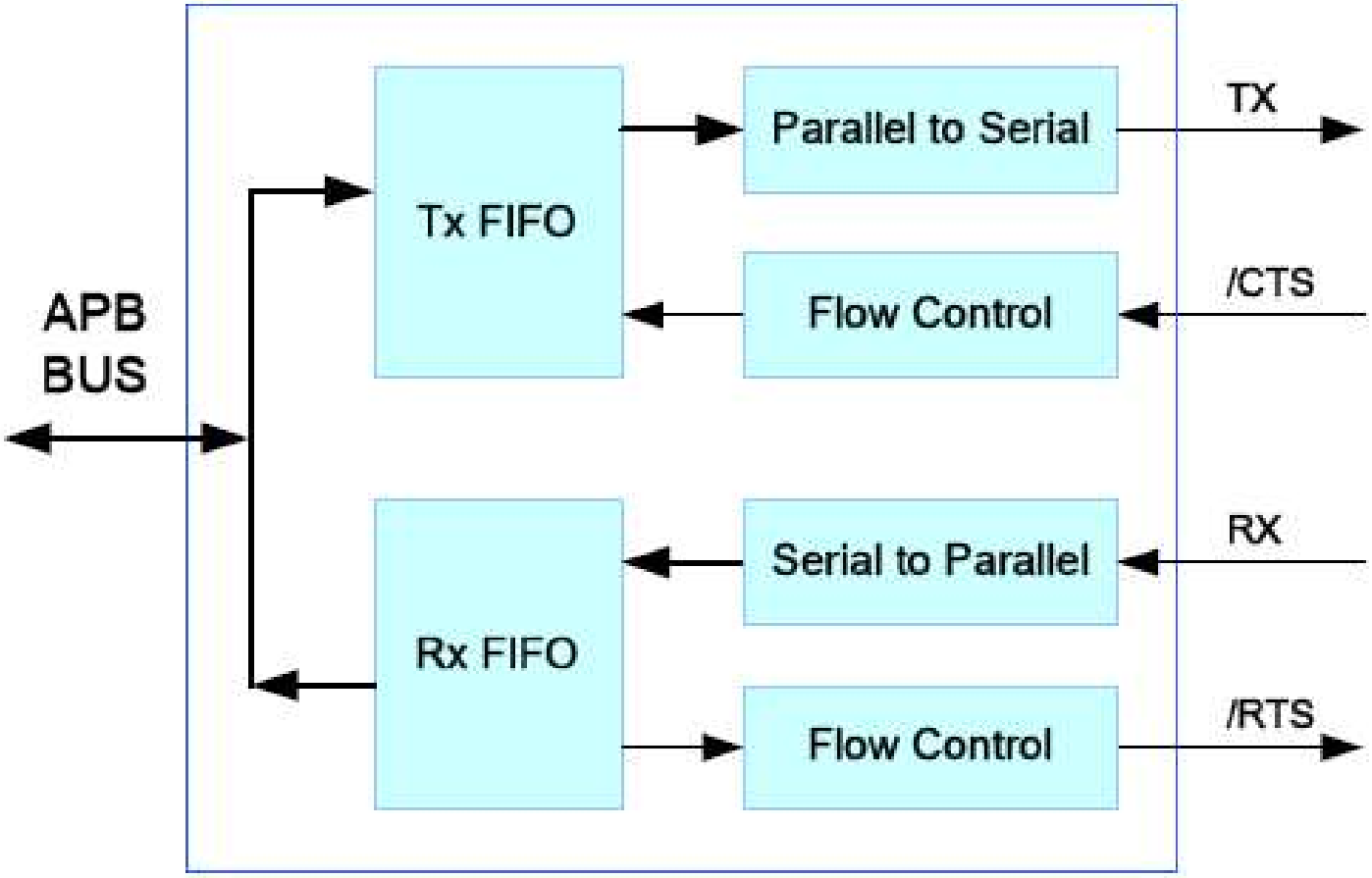
Mode	DIV_X_EN	DIV_X_ONE	Divider X	BRD	Baud rate equation
Mode 0	Disable	0	B	A	UART_CLK / [16 * (A+2)]
Mode 1	Enable	0	B	A	UART_CLK / [(B+1) * (A+2)] , B must >= 8
Mode 2	Enable	1	B	A	UART_CLK / (A+2), A must >=3

UART Transmission Baud Rate

System source = 22.1184MHz			
Baud rate	Mode0	Mode1	Mode2
921600	x	A=0,B=11	A=22
460800	A=1	A=1,B=15 A=2,B=11	A=46
230400	A=4	A=4,B=15 A=6,B=11	A=94
115200	A=10	A=10,B=15 A=14,B=11	A=190
57600	A=22	A=22,B=15 A=30,B=11	A=382
38400	A=34	A=62,B=8 A=46,B=11 A=34,B=15	A=574
19200	A=70	A=126,B=8 A=94,B=11 A=70,B=15	A=1150
9600	A=142	A=254,B=8 A=190,B=11 A=142,B=15	A=2302
4800	A=286	A=510,B=8 A=382,B=11 A=286,B=15	A=4606

UART Auto Flow Control

Department of Electrical Electronic Engineering, UTP



Note: Only supported in UART0 and UART1

RS485 Function Mode

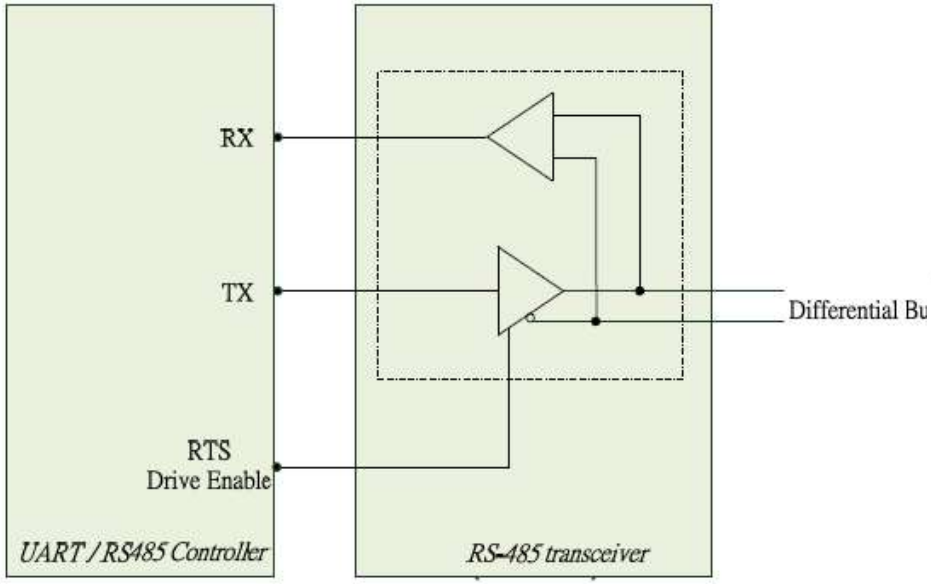
- UART support RS485 9-bit mode function
  - Bit 9 = 1 : Address byte, 0 : Data byte

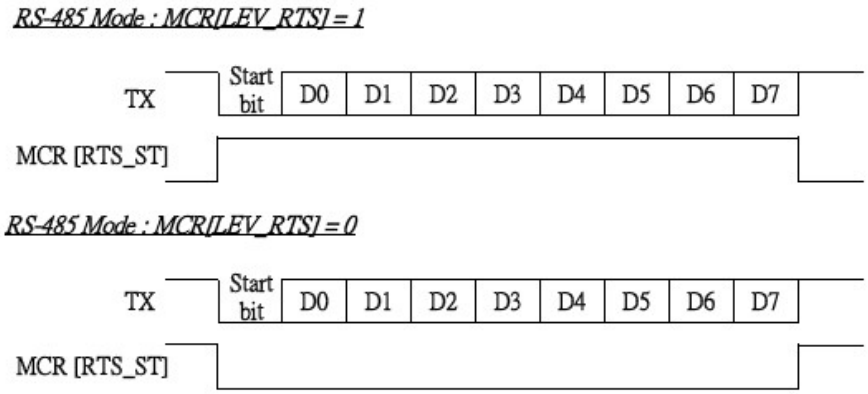
Support 3 operation modes :

- RS-485 Normal Multidrop Operation Mode (NMM)
  - UART_FCR[RS485_RX_DIS] is enabled to ignore data until an address byte is detected (bit9=1)
  - UART_FCR[RS485_RX_DIS] is disabled to receive any data
- RS-485 Auto Address Detection Operation Mode (AAD)
  - The receiver will ignore any data until an address byte is detected (bit9 = 1) and the address byte match the UA_RS-485[ADDR_MATCH] value
- RS-485 Auto Direction Control Operation Model (AUD)
  - Using RTS as output control signal for RS485 transciever
  - LEV_RTS to set RTS active level

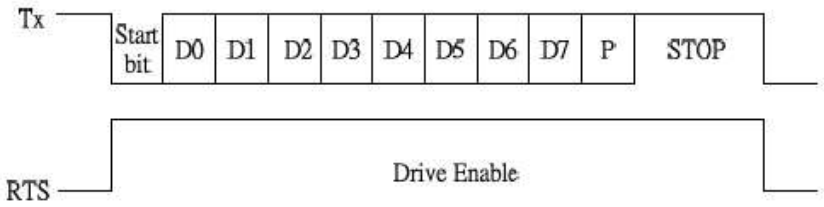
RS485 connected to the transceiver

Department of Electrical Electronic Engineering, UTP





RTS level setting



RTS signal pin as TX OE

UART0 initialization example


```
int32_t main (void)
{
    STR_UART_T param;
    /* GPIO initial */
    DrvGPIO_InitFunction(FUNC_UART0);

     /* Enable and Select UART clock source*/
    UNLOCKREG();
    /* Enable the 12MHz oscillator oscillation */
    DrvSYS_SetOscCtrl(E_SYS_XTL12M, 1);
    /* Waiting for 12M Xtal stalble */
    SysTimerDelay(5000);
    /* HCLK clock source. 0: external 12MHz; 4:internal 22MHz RC oscillator */
    DrvSYS_SetHCLKSource(0);
    LOCKREG();

    /*Open UART0*/
    param.u32BaudRate        = 115200;
    param.u8cDataBits        = DRVUART_DATABITS_8;
    param.u8cStopBits        = DRVUART_STOPBITS_1;
    param.u8cParity          = DRVUART_PARITY_NONE;
    param.u8cRxTriggerLevel  = DRVUART_FIFO_1BYTES;
    param.u8TimeOut          = 0;
    DrvUART_Open(UART_PORT0, &param);
```

Smpl_UART0

- TX at main loop
- RX at UART interrupt handler
- UART connection (digital I/O without transceiver)
  - UART-RX0 connected to another board's TX0
  - UART-TX0 connected to another board's RX0

Smpl_UART0


```
//
// Smpl_UART0 : while loop for UART0-TX keep transmitting 8 bytes string
//            : IRQ routine for UART0-RX keep receiving 8 bytes string & print to LCD
//            : need two learning board to perform UART communication (TX & RX at the same time)
//
// Nu-LB-NUC140
// pin32 GPB0/RX0 to another board's UART TX
// pin33 GPB1/TX0 to another board's UART RX

#include <stdio.h>
#include "NUC1xx.h"
#include "GPIO.h"
#include "SYS.h"
#include "UART.h"
#include "LCD.h"
volatile uint8_t comRbuf[16];
volatile uint16_t comRbytes = 0;
volatile uint16_t comRhead  = 0;
volatile uint16_t comRtail  = 0;

char TEXT1[16] = "TX: sending...  ";
char TEXT2[16] = "RX:             ";
```

Smpl_UART0


```
void UART_INT_HANDLE(void)
{
        uint8_t i;
        uint8_t bInChar[1] = {0xFF};

        while(UART0->ISR.RDA_IF==1)
        {
                DrvUART_Read(UART_PORT0,bInChar,1);
                if(comRbytes < 8) // check if Buffer is full
                {
                        comRbuf[comRbytes] = bInChar[0];
                        comRbytes++;
                }
                else if (comRbytes==8)
                {
                        comRbytes=0;
                        sprintf(TEXT2+4,"%s",comRbuf);
                        print_Line(2,TEXT2);
                }
        }
}
```

Smpl_UART0


```
int32_t main()
{
   uint8_t  i =0;
   uint8_t  dataout[9] = "NuMicro0";
   STR_UART_T sParam;

   UNLOCKREG();
   DrvSYS_Open(48000000);
   LOCKREG();
   init_LCD();
   clear_LCD();
   print_Line(0,"Smpl_UART0    ");

   /* Set UART Pin */
   DrvGPIO_InitFunction(E_FUNC_UART0);

   /* UART Setting */
   sParam.u32BaudRate        = 9600;
   sParam.u8cDataBits        = DRVUART_DATABITS_8;
   sParam.u8cStopBits        = DRVUART_STOPBITS_1;
   sParam.u8cParity          = DRVUART_PARITY_NONE;
   sParam.u8cRxTriggerLevel= DRVUART_FIFO_1BYTES;
```

```
  /* Set UART Configuration */
    if(DrvUART_Open(UART_PORT0,&sParam) !=
E_SUCCESS);
  DrvUART_EnableInt(UART_PORT0, DRVUART_RDAINT,
UART_INT_HANDLE);
  while(1)
  {
    dataout[7] = 0x30 +i;
    DrvUART_Write(UART_PORT0, dataout,8);
    i++;
    if (i>=10) i = 0;
    sprintf(TEXT1+14,"%d",i);
    print_Line(1,TEXT1);
    DrvSYS_Delay(5000000);
  }
  //DrvUART_Close(UART_PORT0);
}
```

Ultrasound Sensor (SRF04 / US100)

- SRF04/US100 ultrasound sensor can range 2cm~4.5m distance, wide operating voltage range at 2.4~5.5V, low static current <2mA, built-in temperature sensor for self-compensation of distance, GPIO/UART as interface
- Pin Connections
  - Pin 1 : Vcc
  - Pin 2 : TX to NUC140 TX
  - Pin 3 : RX to NUC140 RX
  - Pin 4 : Gnd to Gnd
  - Pin 5 : Gnd to Gnd





Smpl_UART0_SRF04

Department of Electrical Electronic Engineering, UTP



Smpl_UART0_SRF04


```
int32_t main()
{
uint32_t distance;
uint8_t  write_byte[1];
uint8_t  read_byte[2];
  char TEXT[16] = "               ";
  STR_UART_T sParam;
  UNLOCKREG();
  DrvSYS_Open(48000000);
  LOCKREG();

  DrvGPIO_InitFunction(E_FUNC_UART0); // Set UART pins

  /* UART Setting */
  sParam.u32BaudRate        = 9600;
  sParam.u8cDataBits        = DRVUART_DATABITS_8;
  sParam.u8cStopBits        = DRVUART_STOPBITS_1;
  sParam.u8cParity          = DRVUART_PARITY_NONE;
  sParam.u8cRxTriggerLevel = DRVUART_FIFO_1BYTES;

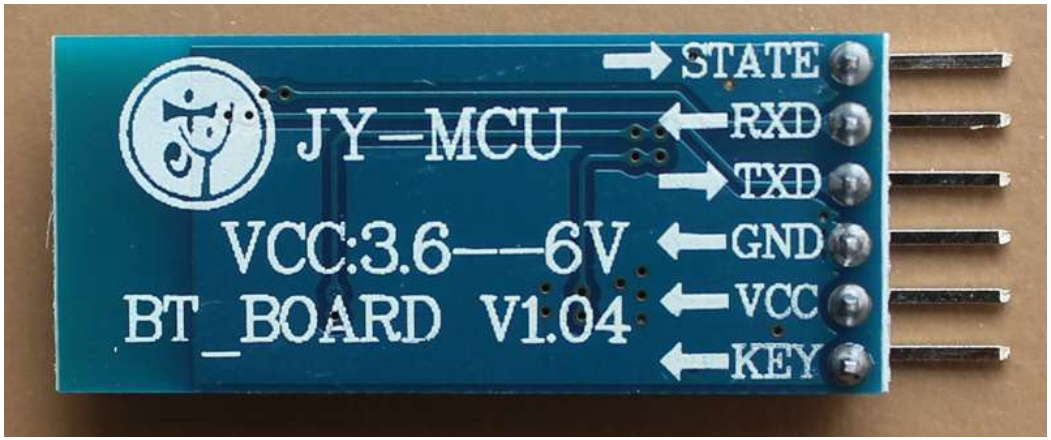
  /* Set UART Configuration */
   if(DrvUART_Open(UART_PORT0,&sParam) != E_SUCCESS);
init_LCD(); // initialize LCD panel
clear_LCD(); // clear LCD panel
print_Line(0, "Smpl_UART0_SRF04");
while(1)
  {
  write_byte[0]=0x55; // trigger SRF04
  DrvUART_Write(UART_PORT0,write_byte,1);
  DrvUART_Read(UART_PORT0,read_byte,2);
  distance = read_byte[0]*256 + read_byte[1];
  sprintf(TEXT+0,"Distance: %dmm",distance);
  print_Line(1, TEXT);
  }

}
```

Bluetooth module

Department of Electrical Electronic Engineering, UTP





Connections
N.C.
MCU TX
MCU RX
GND
VCC
N.C.

HC-05 can operate at 5V

Smpl_UART0_HC05

Department of Electrical Electronic Engineering, UTP



Smpl_UART0_HC05

Department of Electrical Electronic Engineering, UTP

```
#define DATASIZE 8

int32_t main()
{
   uint8_t  read_buf[16] = "               ";
   char     TEXT[16]     = "               ";
   STR_UART_T sParam;

   UNLOCKREG();
   DrvSYS_Open(48000000);
   LOCKREG();

   DrvGPIO_InitFunction(E_FUNC_UART0); // Set UART pins

   /* UART Setting */
   sParam.u32BaudRate        = 9600;
   sParam.u8cDataBits        = DRVUART_DATABITS_8;
   sParam.u8cStopBits        = DRVUART_STOPBITS_1;
   sParam.u8cParity          = DRVUART_PARITY_NONE;
   sParam.u8cRxTriggerLevel= DRVUART_FIFO_1BYTES;
```

```
/* Set UART Configuration */
if(DrvUART_Open(UART_PORT0,&sParam) != E_SUCCESS);

init_LCD();              // initialize LCD panel
clear_LCD();             // clear LCD panel

print_Line(0, "Smpl_UART0_HC05"); // print title

 while(1) {
// read 8 bytes from bluetooth
 DrvUART_Read(UART_PORT0,read_buf,DATASIZE);
 sprintf(TEXT,"%s",read_buf); // display on LCD
 print_Line(1, TEXT);
 }

}
```

Note: Smartphone app can transmit bytes as command to MCU, then control output

Ultrasound Sensor + MCU + Bluetooth

Department of Electrical Electronic Engineering, UTP

MCU read ultrasound sensor, then send data through Bluetooth to PC/Smartphone

- Step1. connecting HC05 & US100 onto Nu-LB-NUC140 (UART port **0 & 2**)
- Step2. build & download Smpl_UART_BT_US100 code to NUC140
- Step3. add bluetooth device to PC/Smartphone
  - HC-05 pairing password = 1234
- Step4. open Terminal on PC/Smartphone
  - Terminal set baud rate = 9600, 8-bit data, no parity, no flow-control
  - Connect to BT device
- Step5. see data displaying on terminal

Note: UART1 connected to LIN circuit which make it get wrong reading from US100

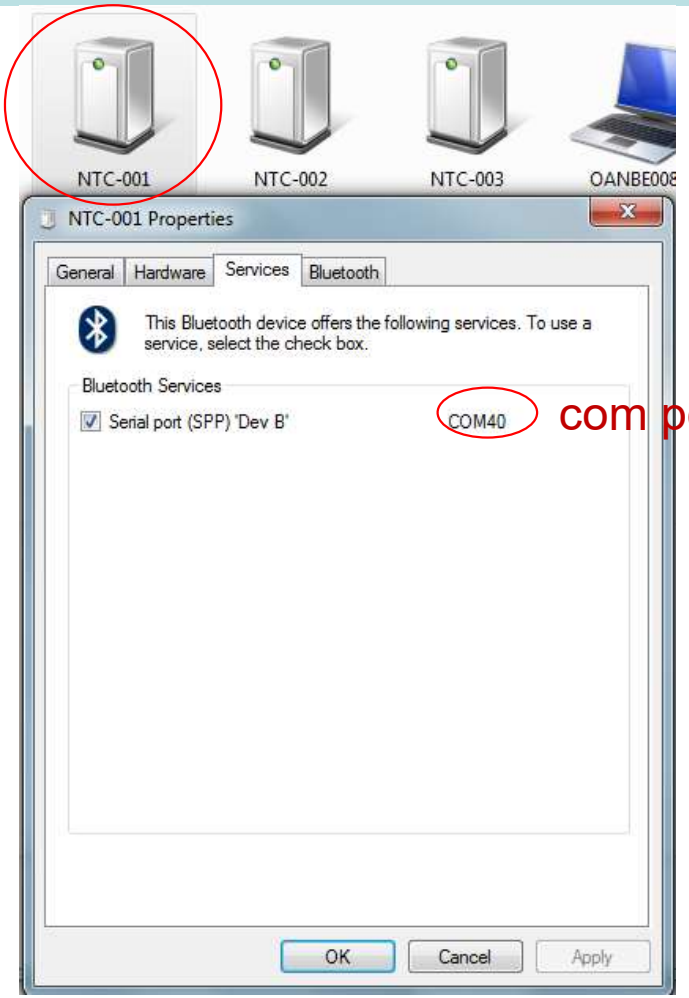
Smpl_UART0_HC05_US100

Department of Electrical Electronic Engineering, UTP



Add Bluetooth Device

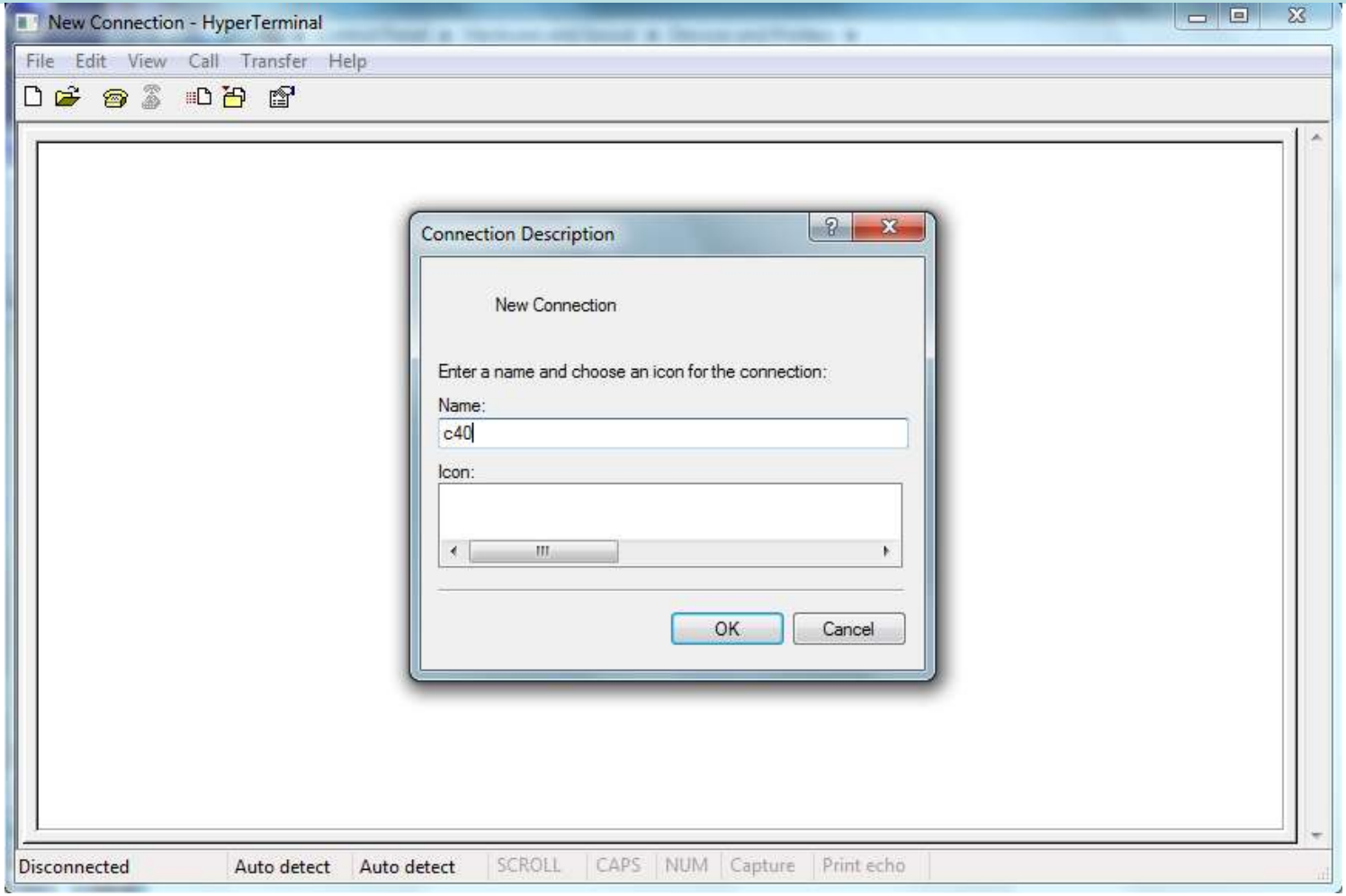
device added



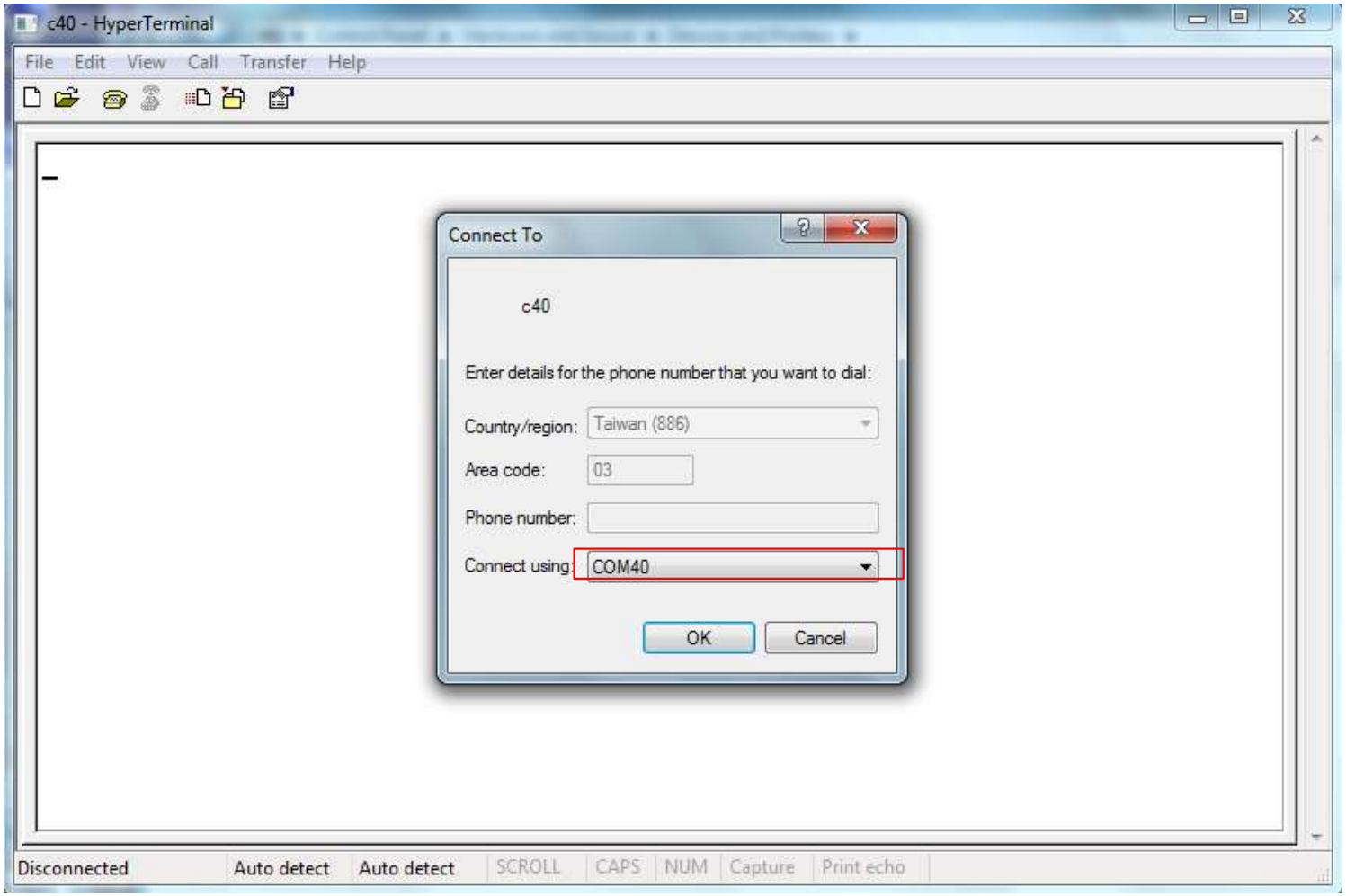
com port no.

Note: after the device is added, check its property to find its com port number

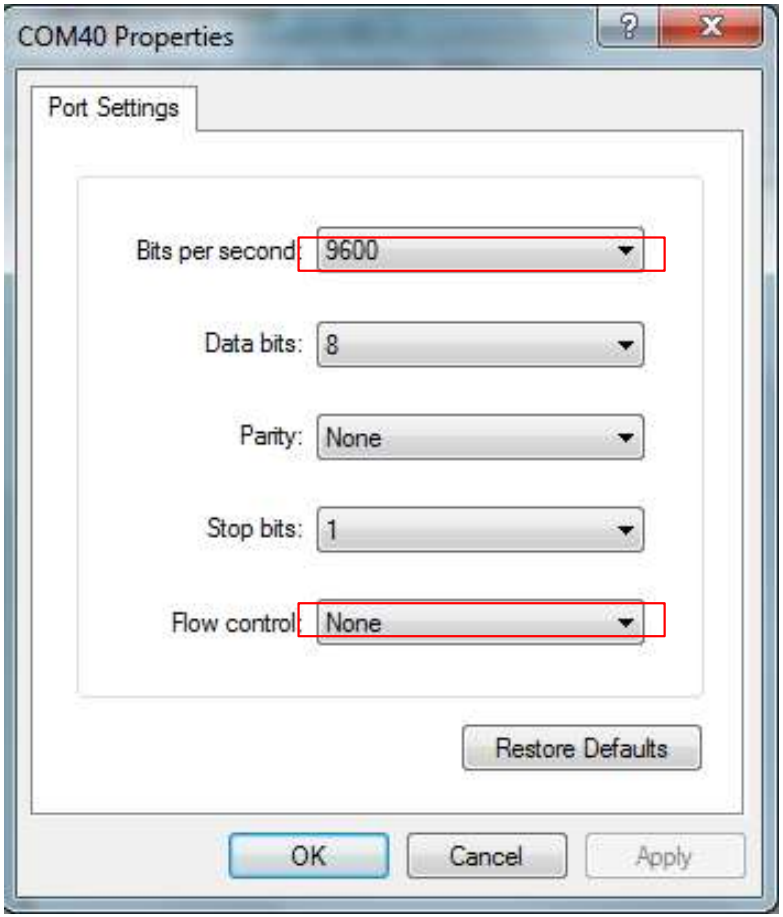
Open PC HyperTerminal



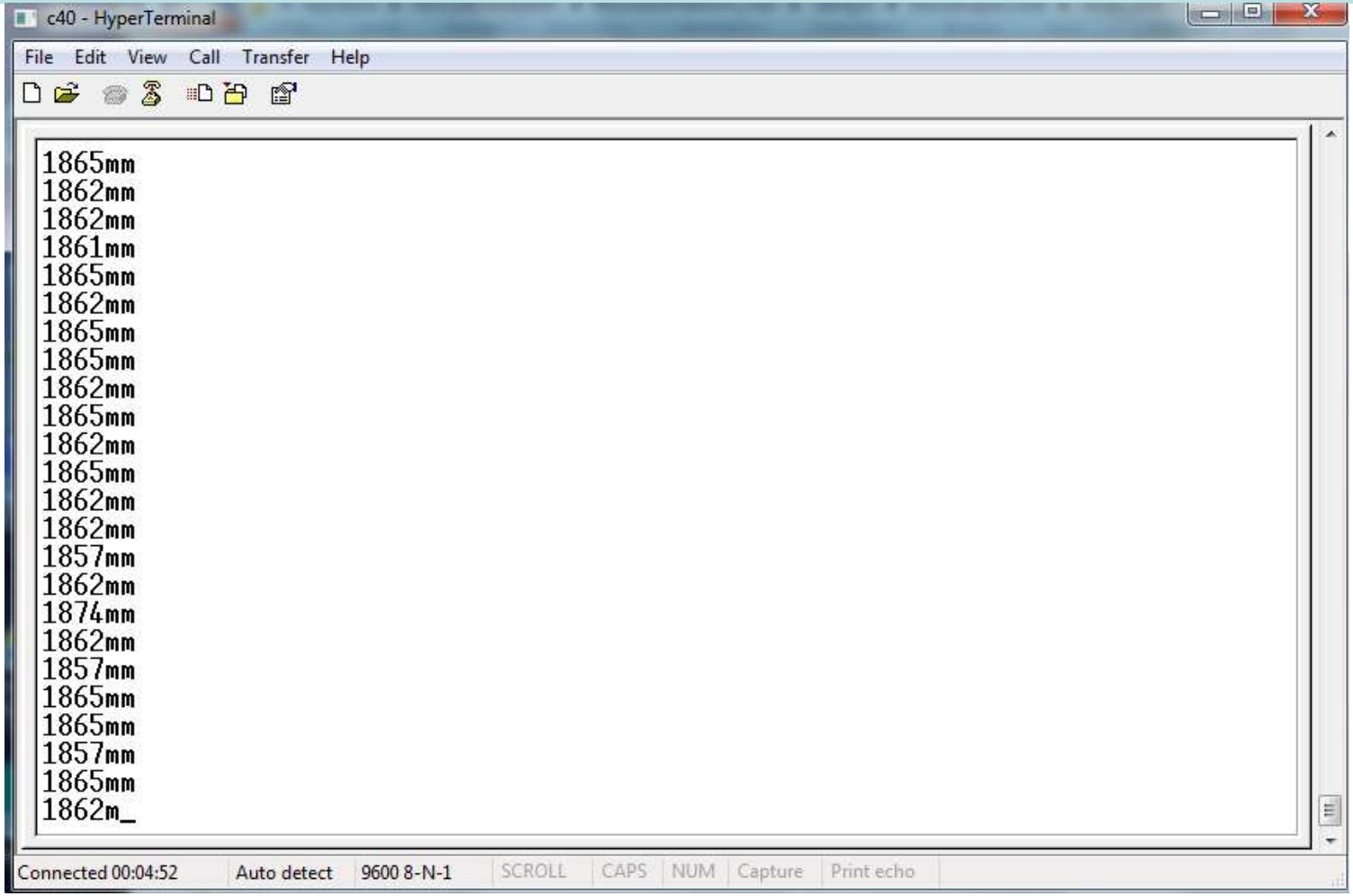
Select HyperTerminal connecting com port



Setup com port parameter



See output on HyperTerminal



Smpl_UART_HC05_US100


```
#define  DATASIZE 8

int32_t main()
{
   uint16_t distance;
   uint8_t  write_byte[1];
   uint8_t  read_byte[2];
   uint8_t  dataout[DATASIZE];

   STR_UART_T sParam;

   UNLOCKREG();
   DrvSYS_Open(48000000);
   LOCKREG();
   init_LCD();
   clear_LCD();
   print_Line(0,"Smpl _BT_US100");
   print_Line(1,"UART2 to US100");
   print_Line(2,"UART0 to HC-05");
   print_Line(3,"PC HyperTerm  ");
```

```
/* Set UART Pin */
DrvGPIO_InitFunction(E_FUNC_UART0);
DrvGPIO_InitFunction(E_FUNC_UART2);

/* UART Setting */
   sParam.u32BaudRate      = 9600;
   sParam.u8cDataBits      = DRVUART_DATABITS_8;
   sParam.u8cStopBits      = DRVUART_STOPBITS_1;
   sParam.u8cParity        = DRVUART_PARITY_NONE;
   sParam.u8cRxTriggerLevel= DRVUART_FIFO_1BYTES;

/* Set UART Configuration */
if(DrvUART_Open(UART_PORT0,&sParam) != E_SUCCESS);
if(DrvUART_Open(UART_PORT2,&sParam) != E_SUCCESS);

while(1)
{
   write_byte[0]=0x55; // trigger SRF04
   DrvUART_Write(UART_PORT2,write_byte,1);
   DrvUART_Read(UART_PORT2,read_byte,2);
```

Smpl_UART0_HC05_US100


```
distance = read_byte[0]*256 + read_byte[1];

dataout[0] = 0x30 + distance /1000;
distance   = distance - distance /1000 * 1000;
dataout[1] = 0x30 + distance /100;
distance   = distance - distance /100 * 100;
dataout[2] = 0x30 + distance/10;
distance   = distance - distance /10 * 10;
dataout[3] = 0x30 + distance;
dataout[4] = 'm';
dataout[5] = 'm';
dataout[6] = 0x0d;
dataout[7] = 0x0a;

DrvUART_Write(UART_PORT0,dataout,DATASIZE);     // send
to Bluetooth
}

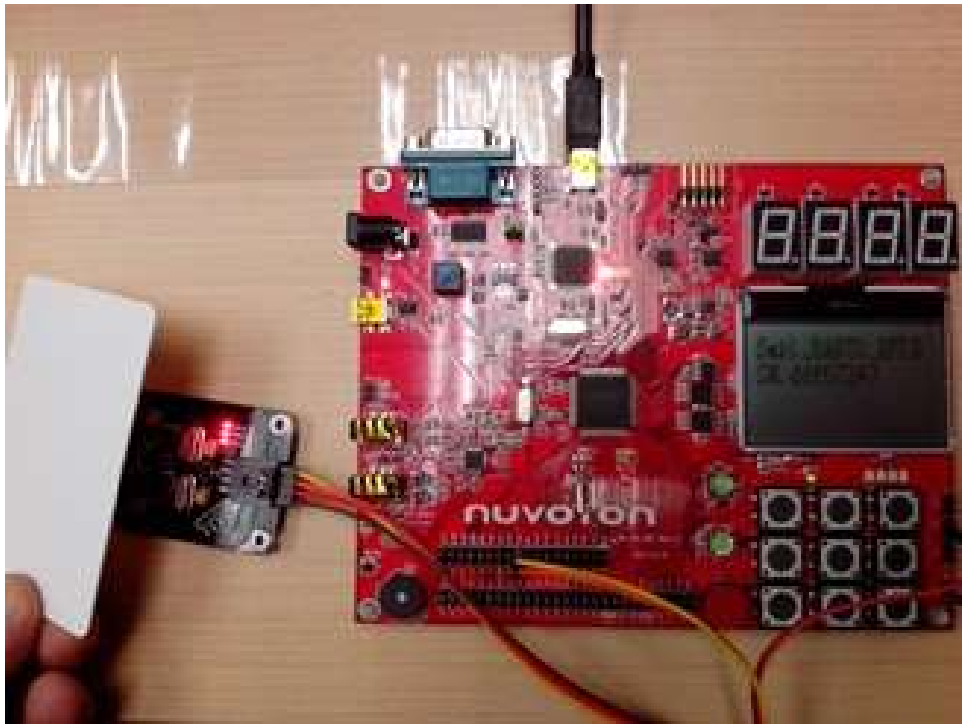
}
```

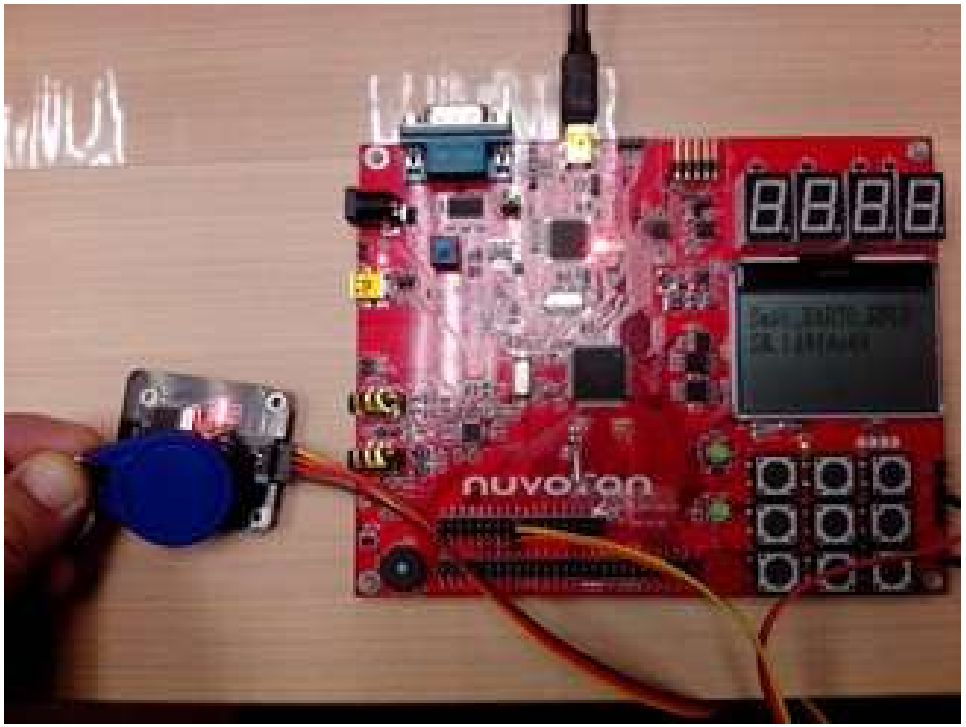
Smpl_UART0_HC05_MPU6050

Department of Electrical Electronic Engineering, UTP

Smpl_UART0_RFID (RFID Reader)

Department of Electrical Electronic Engineering, UTP





Important Notice !

- **This educational material are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".**
- **Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.**
- **All Insecure Usage shall be made at user's own risk, and in the event that third parties lay claims to the author as a result of customer's Insecure Usage, the user shall indemnify the damages and liabilities thus incurred by using this material.**
- **Please note that all lecture and sample codes are subject to change without notice. All the trademarks of products and companies mentioned in this material belong to their respective owners.**